start compiling

start compiling

It can be seen that when the Release_to_customer.sh script is executed, 3 parameters (-f, -p, -q) need to be passed;

- **-f indicates the type of flash, optional nand, nor;**
- **-p indicates the chip model, optional ssd201, ssd202;**
- **-q means fast boot mode, optional fastboot or empty;**
- **-o means to select the corresponding development board configuration, optional 2DO6 or 2D07;**
  2D06: dual network port configuration; 2D07: 7-inch development board configuration

```
while getopts "f:p:q:o:" opt; do
  case $opt in
    f)
      flashtype=$OPTARG
      ;;
    p)
      chip=$OPTARG
      ;;
    q)
      fastboot=$OPTARG
      ;;
    o)
      project=$OPTARG
      ;;
    \?)
      echo "Invalid option: -$OPTARG" >&2
      ;;
  esac
done
```

Here, take nand+ssd201 as an example to start compiling the source code:

```
# ./Release_to_customer.sh -f nand -p ssd201  -o 2D07
```

After the compilation is completed, a system image will be generated in the images directory, and then we can burn these images into the chip through the operations in Chapter 3.

```
ronnie@wt_rd_server:~/work/ssd201/ssd20x_sdk_v008$ ls images
appconfigs.ubifs  boot     customer.ubifs  ipl_s.bin  logo            rootfs.ubifs  uboot_s.bin
auto_update.txt   cis.bin  ipl_cust_s.bin  kernel     miservice.ubifs  scripts
ronnie@wt_rd_server:~/work/ssd201/ssd20x_sdk_v008$
```

After compiling once, if you do not change the chip model, you can comment the makeclean of uboot and kernel in Release_to_customer.sh.

```
# build uboot
cd ${RELEASEDIR}/boot
declare -x ARCH="arm"
declare -x CROSS_COMPILE="arm-linux-gnueabihf-"
if [ "${flashtype}" = "nor" ]; then
    make infinity2m_defconfig
else
    make infinity2m_spinand_defconfig
fi
#make clean
make -j8

if [ "${flashtype}" = "nor" ]; then
    if [ -d ../project/board/i2m/boot/nor/uboot ]; then
        cp u-boot.xz.img.bin ../project/board/i2m/boot/nor/uboot
    fi
else
    if [ -d ../project/board/i2m/boot/spinand/uboot ]; then
        cp u-boot_spinand.xz.img.bin ../project/board/i2m/boot/spinand/uboot
    fi
fi

#build kernel
cd ${RELEASEDIR}/kernel
declare -x ARCH="arm"
declare -x CROSS_COMPILE="arm-linux-gnueabihf-"
if [ "${flashtype}" = "nor" ]; then
    if [ "${fastboot}" = "fastboot" ]; then
        make infinity2m_ssc011a_s01a_fastboot_defconfig
    else
        make infinity2m_ssc011a_s01a_minigui_defconfig
    fi

else
    if [ "${fastboot}" = "fastboot" ]; then
        make infinity2m_spinand_ssc011a_s01a_minigui_fastboot_defconfig
    else
        #make infinity2m_spinand_ssc011a_s01a_minigui_defconfig
        make ${KERNEL_DEFCONFIG}
    fi

fi
#make clean
make -j8
```

After successfully compiling once, you can modify the compilation script according to your own situation, otherwise the modified configuration will be overwritten every time you compile, which will cause your own configuration to fail.

Take spinand and the configuration of the kernel as an example, make the following modifications to the Release_to_customer.sh script: **(Method 1 and Method 2 are both feasible)**

```
#build kernel
cd ${RELEASEDIR}/kernel
declare -x ARCH="arm"
declare -x CROSS_COMPILE="arm-linux-gnueabihf-"
if [ "${flashtype}" = "nor" ]; then
    if [ "${fastboot}" = "fastboot" ]; then
        make infinity2m_ssc011a_s01a_fastboot_defconfig
    else
        make infinity2m_ssc011a_s01a_minigui_defconfig
    fi

else
    if [ "${fastboot}" = "fastboot" ]; then
        echo fast kernel
        make infinity2m_spinand_ssc011a_s01a_minigui_fastboot_defconfig
    else
        #Method 1:
        cp .config infinity2m_spinand_ssc011a_s01a_minigui_defconfig

        #Method 2:
        #.config

        #make infinity2m_spinand_ssc011a_s01a_minigui_defconfig
    fi

fi
```

Document update time: 2021-10-19 18:36 Author: Aeeditor